

Band Mask to Resistor Value Conversion Block

Automated Resistor Color Code Reader — ECEN 491 Undergraduate Research

Osama Hasoneh

Mentor: Dr. Mohammad Shaqfeh

Texas A&M University at Qatar

Department of Electrical & Computer Engineering

ECEN 491 — Undergraduate Research

Fall 2025

Outline

- 1 Problem Statement
- 2 Our Contribution: Band Mask to Resistor Value
- 3 Pipeline Architecture
- 4 Algorithm Details
- 5 Color Code Mapping
- 6 Data Structures
- 7 End-to-End Walkthrough
- 8 Results & Validation
- 9 Robustness Features
- 10 Limitations & Future Work
- 11 Summary

The Problem

Challenge

Manually reading resistor color bands is:

- **Time-consuming** — requires memorizing the color code table
- **Error-prone** — similar colors (brown/red, orange/yellow) easily confused
- **Orientation-dependent** — reading direction must be determined correctly
- **Not scalable** — does not scale for production or lab environments

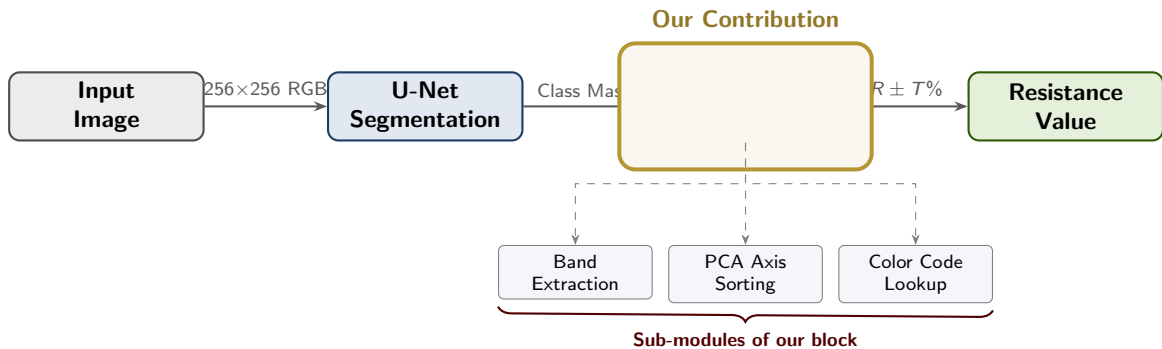
Goal

Automate the process: **Image** → **Resistance Value**



How do we automate this?

Overall System Architecture



Our Contribution — Overview

What We Built

The **Band Mask to Resistor Value** conversion block — the complete post-segmentation pipeline that converts a pixel-level class mask into a resistance value.

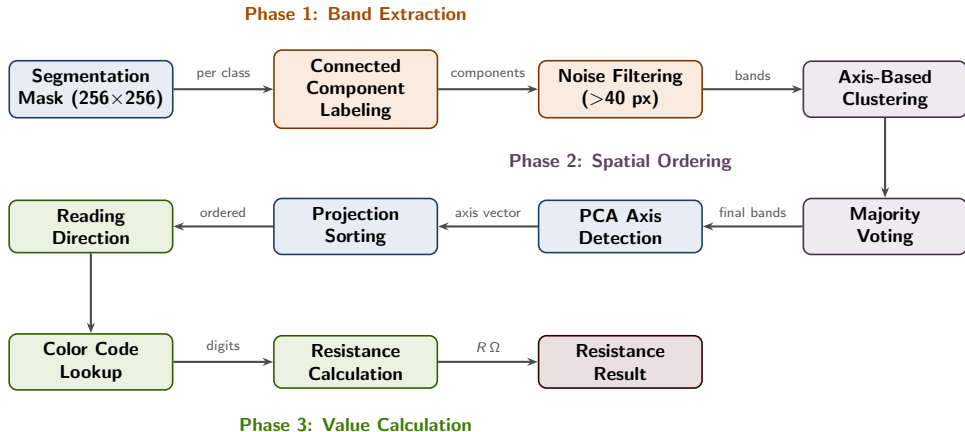
Key Modules (2,650 Lines of Python)

- 1 `band_extractor.py` — 816 lines
- 2 `resistance_calculator.py` — 460 lines
- 3 `axis_detector.py` — 197 lines
- 4 `color_code_tables.py` — 229 lines
- 5 `run_inference.py` — 470 lines
- 6 `validate_results.py` — 267 lines

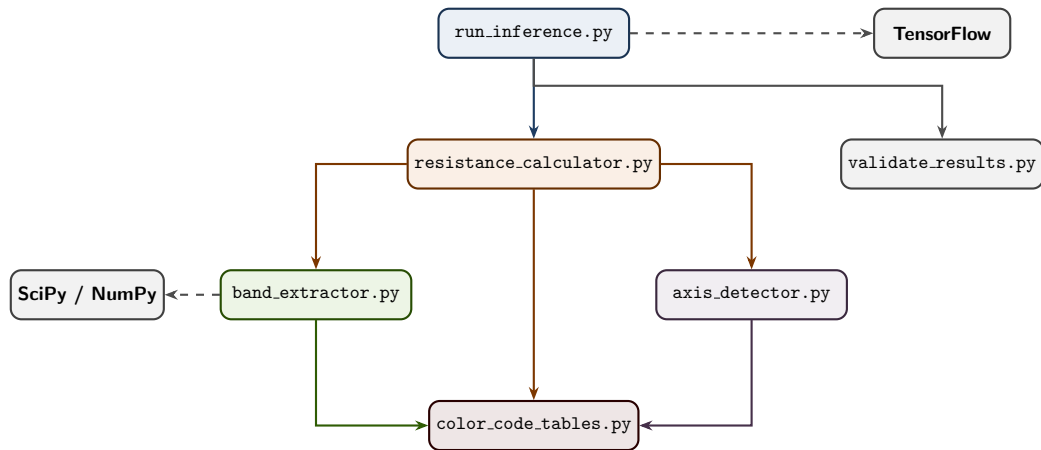
Capabilities

- Handles **3, 4, and 5-band** resistors
- Supports **arbitrary orientations** via PCA
- Automatic **reading direction** detection using gold tolerance band
- Robust **noise filtering** via connected component analysis
- Axis-based **majority voting** for conflicting regions
- **Validation framework** with E24 series checking

Detailed Pipeline Flowchart



Module Architecture

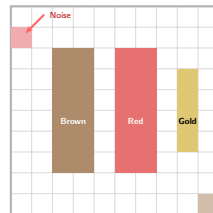


Step 1: Band Extraction via Connected Component Analysis

Algorithm

For each color class $c \in \{1, \dots, 12\} \setminus \{0, 5\}$:

- 1 Binary mask: $M_c[i, j] = \mathbf{1}[\text{mask}[i, j] = c]$
- 2 Apply `scipy.ndimage.label()` (8-connectivity)
- 3 Per component k : compute centroid, area A_k , bounding box
- 4 **Filter**: Discard if $A_k < 40$ pixels
- 5 **Merge**: Combine nearby same-color fragments (< 15 px)



Mask with noise pixels

CCL separates bands from noise; area filtering removes artifacts.

Step 2: PCA-Based Axis Detection & Sorting

PCA Algorithm

Given centroids $\{\mathbf{p}_1, \dots, \mathbf{p}_n\}$:

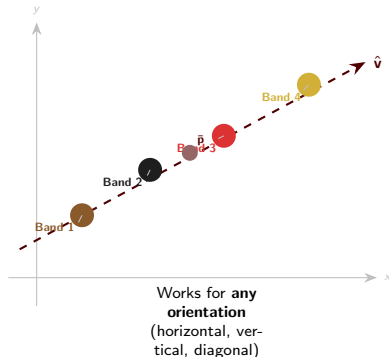
- 1 Center: $\bar{\mathbf{p}} = \frac{1}{n} \sum_i \mathbf{p}_i$
- 2 Covariance: $\Sigma = \frac{1}{n} \mathbf{X}^T \mathbf{X}$
- 3 Eigendecomposition: $\Sigma \mathbf{v} = \lambda \mathbf{v}$
- 4 Principal axis: $\hat{\mathbf{v}} = \text{eigvec}(\max \lambda)$

Projection Sorting

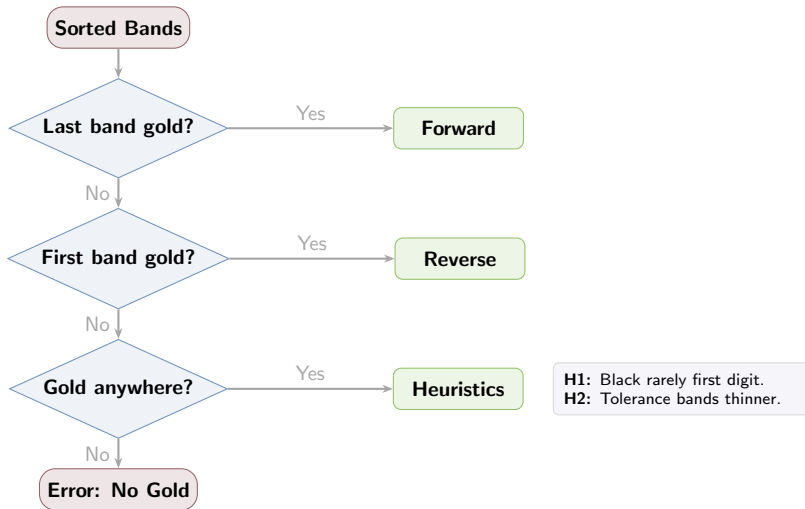
For each band i , compute scalar projection:

$$s_i = (\mathbf{p}_i - \bar{\mathbf{p}}) \cdot \hat{\mathbf{v}}$$

Sort bands by s_i to obtain left-to-right order.



Step 3: Reading Direction Detection



Step 4: Resistance Value Calculation

4-Band Resistor

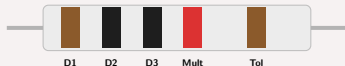


$$R = (D_1 \times 10 + D_2) \times \text{Mult}$$

Ex: Brown-Black-Red-Gold

$$R = (1 \times 10 + 0) \times 100 = 1 \text{ k}\Omega \pm 5\%$$

5-Band Resistor (Precision)



$$R = (D_1 \times 100 + D_2 \times 10 + D_3) \times \text{Mult}$$

Ex: Brown-Black-Black-Red-Brown

$$R = (100 + 0 + 0) \times 100 = 10 \text{ k}\Omega \pm 1\%$$

Decision: 3 bands → default tol. | 4 → standard | 5 → precision | 6+ → first 5

Resistor Color Code Reference

Color	Class ID	Digit	Multiplier	Tolerance
Black	8	0	$\times 1$	—
Brown	4	1	$\times 10$	$\pm 1\%$
Red	11	2	$\times 100$	$\pm 2\%$
Orange	2	3	$\times 1k$	—
Yellow	7	4	$\times 10k$	—
Green	3	5	$\times 100k$	$\pm 0.5\%$
Blue	6	6	$\times 1M$	$\pm 0.25\%$
Violet	12	7	$\times 10M$	$\pm 0.1\%$
Grey	10	8	$\times 100M$	$\pm 0.05\%$
White	9	9	$\times 1G$	—
Gold	1	—	$\times 0.1$	$\pm 5\%$

Note: The U-Net model outputs 13 classes — 11 colors + 2 background classes (image background & resistor body).

Core Data Structures

BandInfo

```
@dataclass
class BandInfo:
    class_id: int          # 0-12
    color_name: str        # "brown"
    centroid: (float, float)
    area: int
    bounding_box: (int, int, int, int)
```

CalculationError

```
@dataclass
class CalculationError:
    error_type: str
    message: str
    detected_bands: List[BandInfo]
```

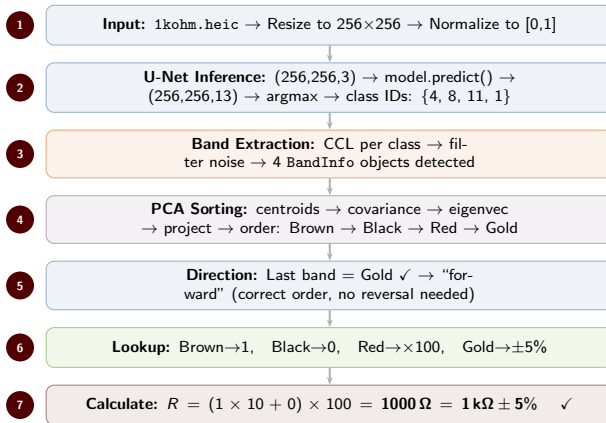
ResistanceResult

```
@dataclass
class ResistanceResult:
    value: float           # e.g. 1000.0
    tolerance: float       # e.g. 5.0
    band_count: int        # 3, 4, or 5
    bands: List[BandInfo]
```

Lookup Tables

```
COLOR_TO_DIGIT = {
    "black":0, "brown":1, "red":2,
    "orange":3, "yellow":4,
    "green":5, "blue":6, "violet":7,
    "grey":8, "white":9 }
COLOR_TO_MULTIPLIER = {
    "black":1, "brown":10,
    "red":100, "orange":1e3, ... }
```

Worked Example: 1 k Ω Resistor



Results & Performance

Test Dataset

- 45 resistor images (HEIC, JPG, PNG)
- Range: 1 Ω to 1 M Ω
- Various orientations and lighting
- 9 validation runs performed

Overall Accuracy

Correct: 69%

21% 10%

Incorrect Error

Performance Metrics

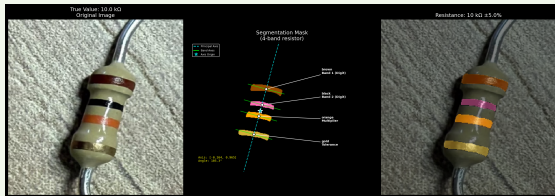
Metric	Value
Images Tested	42
Correct	29 (69%)
Incorrect	9 (21%)
Errors	4 (10%)
Inference Time	~0.5–1 s
Post-processing	~10 ms
Model Memory	~90 MB

Error Sources

- Color confusion (green/grey, brown/black)
- Light reflection mistaken for grey
- Small artifacts mistaken for full bands

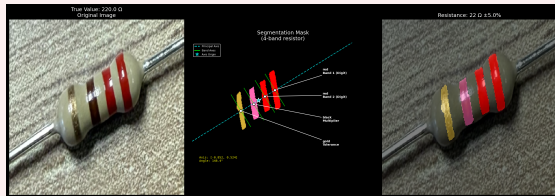
Case Study: Correct vs. Incorrect Conversion

Correct: 10 k Ω Resistor



True Value: 10.0 k Ω
Detected: Brown–Black–Orange–Gold
Calculated: $(1 \times 10 + 0) \times 1000 = 10 \text{ k}\Omega$
Status: **Correct (0% error)**

Incorrect: 220 Ω Resistor



True Value: 220 Ω
Expected: Red–Red–Brown–Gold
Detected: Red–Red–Black–Gold
Calculated: $(2 \times 10 + 2) \times 1 = 22 \Omega$
Status: **Incorrect (90% error)**

Root cause: Brown multiplier band misclassified as black ($\times 1$ instead of $\times 10$)

Robustness & Edge Case Handling

Noise Rejection

- **Area filtering:** Components < 40 px discarded
- **Component merging:** Same-color fragments merged (< 15 px)
- **Band cap:** If > 6 detected, keep top-5 by area

Orientation Invariance

- PCA handles **any angle**
- No horizontal/vertical assumption
- Works with ≥ 2 centroids

Axis-Based Majority Voting

- When colors overlap same axis position:
 - ① Cluster by perpendicular distance (< 15 px)
 - ② Largest total area wins
- Resolves ambiguous segmentation

Validation Checks

- E24 standard series validation
- Range: $0.1\ \Omega - 100\ \text{M}\Omega$
- Color validity per band position

Fine-Tuneable Parameters

Parameter	Value	Purpose	Justification
MIN_BAND_AREA	40 px	Min. pixels for a valid band	Filters noise; tested vs. 50 px
MERGE_DISTANCE	10 px	Max. centroid gap to merge fragments	Reunites splits without merging neighbors
BAND_AXIS_THRESH	15 px	Perpendicular distance for axis grouping	Tolerates slight segmentation offset
MAX_BANDS_TRIGGER	6	Band count to trigger area filter	5-band max + 1 noise tolerance
MAX_BANDS_KEPT	5	Bands kept after filtering	Matches 5-band precision resistor
MIN_BAND_COUNT	3	Min. bands for calculation	2 digits + 1 multiplier minimum
WIDTH_RATIO	0.7	Width ratio for direction heuristic	Tolerance bands typically <70% of digit width
DEFAULT_TOL	20%	Fallback when no tolerance band found	Standard for 3-band resistors
INPUT_SIZE	256×256	Image preprocessing resolution	U-Net training resolution
VALIDATION_TOL	10%	Error threshold for “correct” label	Covers E24 rounding & measurement drift

All values empirically chosen on 45-image dataset — grid search could improve accuracy.

Limitations & Future Work

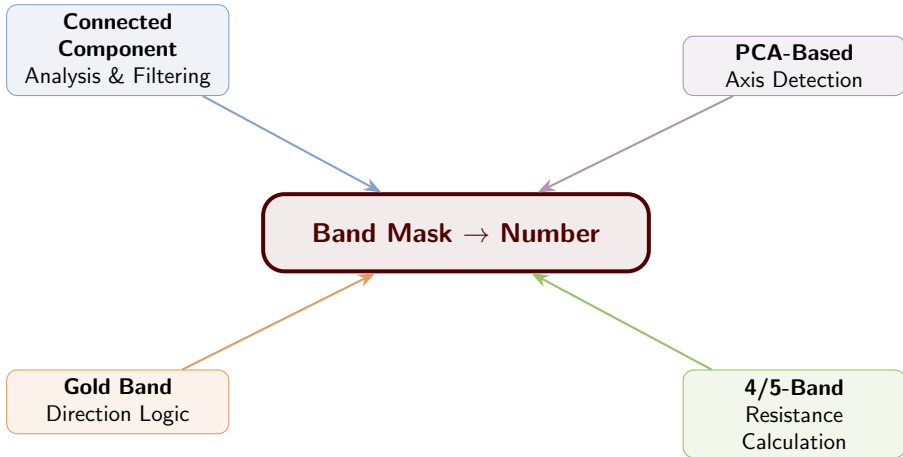
Current Limitations

- **Gold tolerance only** — silver not yet supported
- **No 6-band support** — no temperature coefficient
- **Color confusion** — green/grey, brown/black challenging
- **Light reflection** mistaken for grey band
- **Small artifacts** mistaken for full bands
- **Single resistor** per image only
- **No SMD** resistor support

Future Improvements

- Add **silver tolerance** support
- Implement **6-band** resistors
- Improve **color differentiation**
- Add **confidence scores** per band
- Support **multiple resistors** per image
- Explore **hardware acceleration**
- Expand training dataset

Summary



2,650 lines of Python | 6 modules | 69%
accuracy on 42 images | ~1 s per image

Thank You

Questions?

ECEN 491 — Undergraduate Research
Texas A&M University at Qatar
Fall 2025